BIGFIX PLATFORM

Master Class Diagnostics Workbench

User Guide



VERSIONING

Document Control

Version	Date	Notes
1.2	10-20-2026	Added estimated build times and memory use in 6.3.
1.1	10-14-2026	Added Windows launcher Unblock steps and a file Properties screenshot.
1.0	10-14-2026	User Guide for the Master Class Diagnostics Workbench.

Table of Contents

Before You Start **4**

1.1 Purpose And Project Status 4

1.2 Requirements 4

Setting Up **5**

2.1 Install Python 5

2.2 Install uv 5

2.3 Launch The Workbench 5

2.4 If Setup Fails 7

2.5 Collect The Data 7

Workbench Workflow **8**

Step 1: Folders And Run Details **9**

Step 2: Modules **10**

5.1 Select Modules 10

5.2 Statuses 11

5.3 Client Performance Analysis 11

5.4 Perfmon Infrastructure Analysis 12

5.5 FillDB Performance Analysis 13

Step 3: Build **14**

6.1 Output Terms 15

6.2 The Run Folder 15

6.3 Estimated Build Time And Memory 16

Troubleshooting **17**

Security And Data Handling **18**

8.1 Air-Gapped Analysis 18

8.2 The Private Python Environment 18

8.3 Dependency Cooldowns 18

8.4 Code Sweeps 18

8.5 Data Handling Rules 18

SECTION 01

Before You Start

The Master Class Diagnostics Workbench analyzes files collected from a BigFix deployment. Select one or more analysis modules to create Mini Reports, Infographics, and supporting Excel workbooks in a timestamped run folder.

The Workbench includes three analysis modules:

- **Client Performance Analysis**: client loop evaluation performance, UDP notification coverage, client CPU and hashing settings.
- **Perfmon Infrastructure Analysis**: Windows performance counters assessed against BigFix requirements.
- **FillDB Performance Analysis**: FillDB processing performance.

Collect the input files before you run an analysis. Collection is a separate activity. For a collection summary, see 2.5 Collect The Data. The lab guide provides screenshots and detailed collection steps.

> **NOTE** Analysis reads the collected files locally and writes the results to the output folder you select. See Section 8.

1.1 Purpose And Project Status

Built for the Master Class, the Workbench is a completed class demonstration, not an application under ongoing active development. It is intended to inspire class members and showcase the possibilities of automation.

Class members are encouraged to add their own modules and extend the Workbench. Much of the application was built with AI assistance, and you can continue using AI to develop your own extensions to the framework.

Class members have read-only access to the GitHub repository, https://github.com/dmccalla/BigFix-Master-Class-2026. Make changes in your own copy or fork. Report issues and request enhancements through the repository's Issues tab. Further work on the class version will be considered in response to those requests.

1.2 Requirements

Item	Requirement
Operating system	Windows 11, the documented environment for the Workbench.
Python	Python 3.12 or newer, available through the Python launcher or PATH.
Browser	Microsoft Edge or Google Chrome, used to render Infographics.
Office applications	Microsoft Word and Excel are the preferred applications for opening the outputs.
Folders	A writable application folder and a writable output location.
Collected data	Input files for each module you plan to run. See Section 5 for the supported inputs.
uv	Install after Python. See 2.2 Install uv.

SECTION 02

Setting Up

There is no installer. The Workbench is portable. It runs from the supplied application folder, wherever that folder is placed, as long as the folder structure stays intact.

2.1 Install Python

1. Install Python 3.12 or newer on the Windows computer that runs the Workbench.
   - Make Python available through the Python launcher or PATH. Accept the installer option to add Python to PATH if it is offered.
2. Open a new terminal and confirm the version.
   - Run **py -3 --version** or **python --version**.
   - The version must be 3.12 or newer.

2.2 Install uv

After installing Python, open PowerShell and run:

winget install --id=astral-sh.uv -e

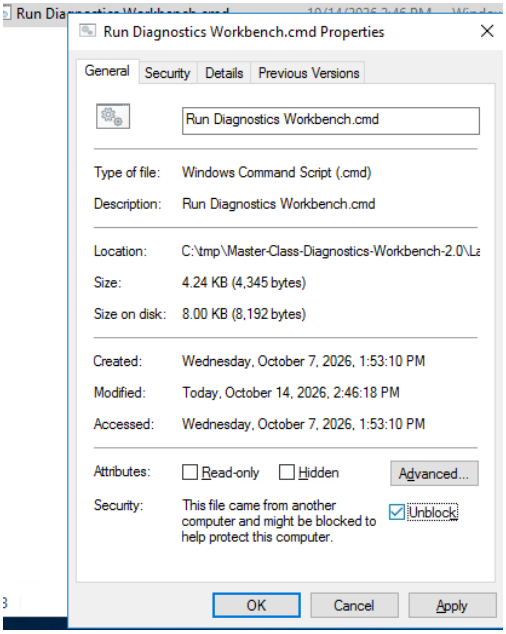
Close and reopen PowerShell. Run uv --version to confirm that uv is available.

2.3 Launch The Workbench

Run **Launchers\Run Diagnostics Workbench.cmd** from the application folder.

Unblock the Windows launcher

Windows may block the .cmd launcher because it was downloaded from the Internet. If it will not run, right-click the .cmd file and select Properties. On the General tab, tick the Unblock box at the bottom, then click OK. Run the launcher again. If Unblock is not shown, no change is needed.



Example: Run Diagnostics Workbench.cmd properties with Unblock selected. The same steps apply to this tool's .cmd launcher.

The first launch creates a private Python environment in a **.venv** folder and installs the packages listed in **requirements.txt** with pip. The first setup needs access to the package source. Later launches use the existing environment.



Figure 1. The first launch builds a private environment, installs packages, and opens the Workbench.

1. Place the supplied application folder in a writable local location.
2. Keep its folder structure intact.
3. Open **Launchers** and double-click **Run Diagnostics Workbench.cmd**.
4. On first launch, acknowledge the notice and allow package installation to finish. Setup time depends on the computer and package access.
5. Continue when the Workbench opens at **Folders**.

> **TIP** Confirm that Microsoft Edge or Google Chrome is installed before you build. Infographics are rendered through the browser.

2.4 If Setup Fails

Problem	Action
The launcher cannot find a supported Python version.	Install Python 3.12 or newer and make it available through the Python launcher or PATH. Run the launcher again.
First setup fails.	Read the launcher error. Check write access to the application folder and access to the package source.
Setup is incomplete.	Close the Workbench. Remove the **.venv** folder from the application folder. Run the launcher again.

2.5 Collect The Data

Run the collection tasks in the Performance and Config Analysis site in the BigFix console. The lab guide provides screenshots and detailed console steps.

Module	Tasks And Highlights
Client Performance Analysis	Activate the BigFix Master Class -- Client Performance Counters - Basic (2.0) analysis first. Normally allow 24 to 48 hours of reporting. BigFix Master Class -- Collect Deployment Information (2.0): run once on the root server before the Client Performance analysis. The Workbench requires its Deployment_Information folder. BigFix Master Class -- Collect Client Performance Counter Analysis Data (2.0): run once on the root server to export the counters, Top 10 evaluations, and UDP notification data.
Perfmon Infrastructure Analysis	BigFix Master Class -- Setup and Run Perfmon Counters (2.0): run on the Windows role servers and SQL hosts in scope. Captures counters every 60 seconds for 24 hours. Retrieve each .blg directly from its server after the capture stops. The tasks do not upload these large files.
FillDB Performance Analysis	BigFix Master Class -- Enable FillDB Performance Logging (2.0): run on the root server for temporary logging. The default duration is 24 hours. Enabling and disabling logging restarts FillDB. Leave automatic disabling enabled and verify its generated action completes. Retrieve the complete log set, including rollover files.

Finish deployment and client/API collection before starting performance captures so collection activity does not distort the measurements.

Keep the Results folder structure intact: Client_Loop, Deployment_Information, Perfmon, and Logs\FillDB_Performance. Select Results as the Data Folder in the Workbench.

The Collect tasks run only after a Master Operator takes the action. They read from the root server through the REST API and contact no other server. They write to BigFix_Master_Class\Platform\Results on the drive that holds the BigFix client. They back up earlier results first and change no BigFix setting.

The Perfmon task creates a data collector set named BFPerf that stays defined after the capture. To remove it run logman stop BFPerf and logman delete BFPerf.

The FillDB task turns on FillDB performance logging in the registry and restarts the FillDB service. The generated disable action restores the original setting. If it does not run, run WARNING: BigFix Master Class -- FillDB Performance Logging is Enabled (2.0).

SECTION 03

Workbench Workflow

The Workbench has three navigation steps.

1. Folders.
2. Modules.
3. Build.

Each build runs the selected modules and writes their outputs to a new run folder. Select **Open Output** to open that folder when the build finishes.

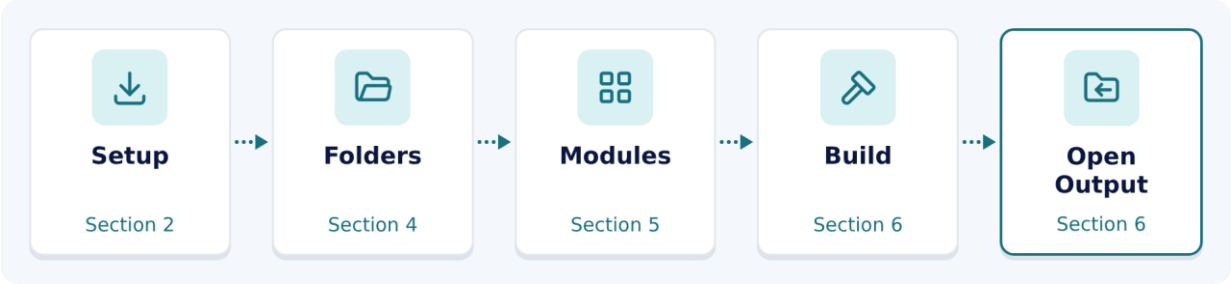


Figure 2. The Workbench workflow, with the guide section for each stage.

Version 2.1 adds dark mode. Use the **Dark mode** switch in the upper-right corner to change between light and dark appearance. The switch is unavailable while a build is running.

SECTION 04

Step 1: Folders And Run Details

STEP 1 Folders	STEP 2 Modules	STEP 3 Build

Step 1 identifies the collected data, the output location, and the run name.

Field	Action
Data Folder	Optional. Select the folder containing the collected files to scan for module inputs. Leave it blank to choose inputs individually on Step 2.
Output Folder	Required. Select a writable location for the run folders.
Run Name	Required. Enter a recognizable name, such as **Example_Server**. It is used in the run-folder name and output filenames.

1. Select the **Data Folder**, or leave it blank.
2. Select the **Output Folder**.
3. Enter the **Run Name**.
4. Select **Next**.

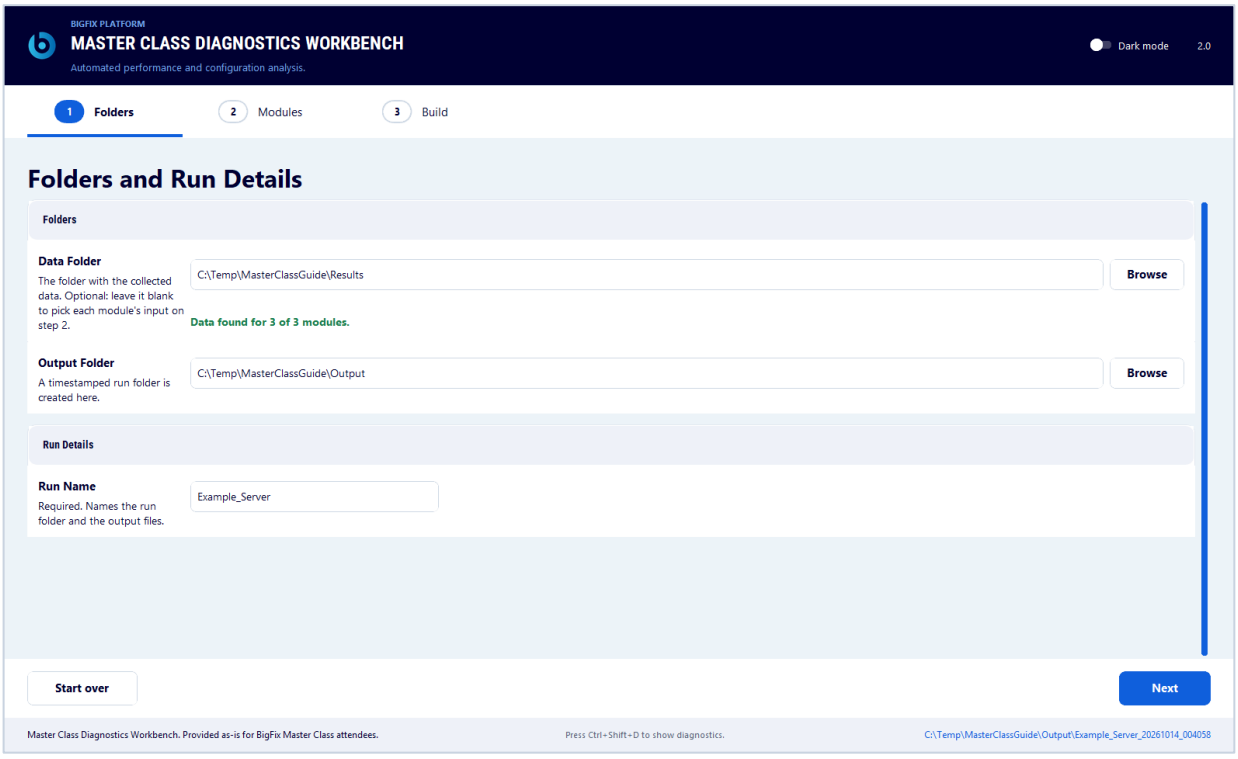


Figure 1. Step 1 with the Data Folder, Output Folder, and Run Name completed.

When a Data Folder is selected, the scan summary reports the inputs found for the three modules. An input missing for a module you do not select does not need to be supplied.

SECTION 05

Step 2: Modules

STEP 1 Folders	STEP 2 Modules	STEP 3 Build

Step 2 lists the three analysis modules. Select the modules to run, check their inputs, and complete their required values. Unselected modules are skipped.

5.1 Select Modules

1. Select the checkbox for each module to run.
2. Open **Details** for each selected module.
3. Check the selected input and complete the required values.
4. Resolve **Needs A File** and **Needs Review** for the selected modules.
5. Select **Next**.

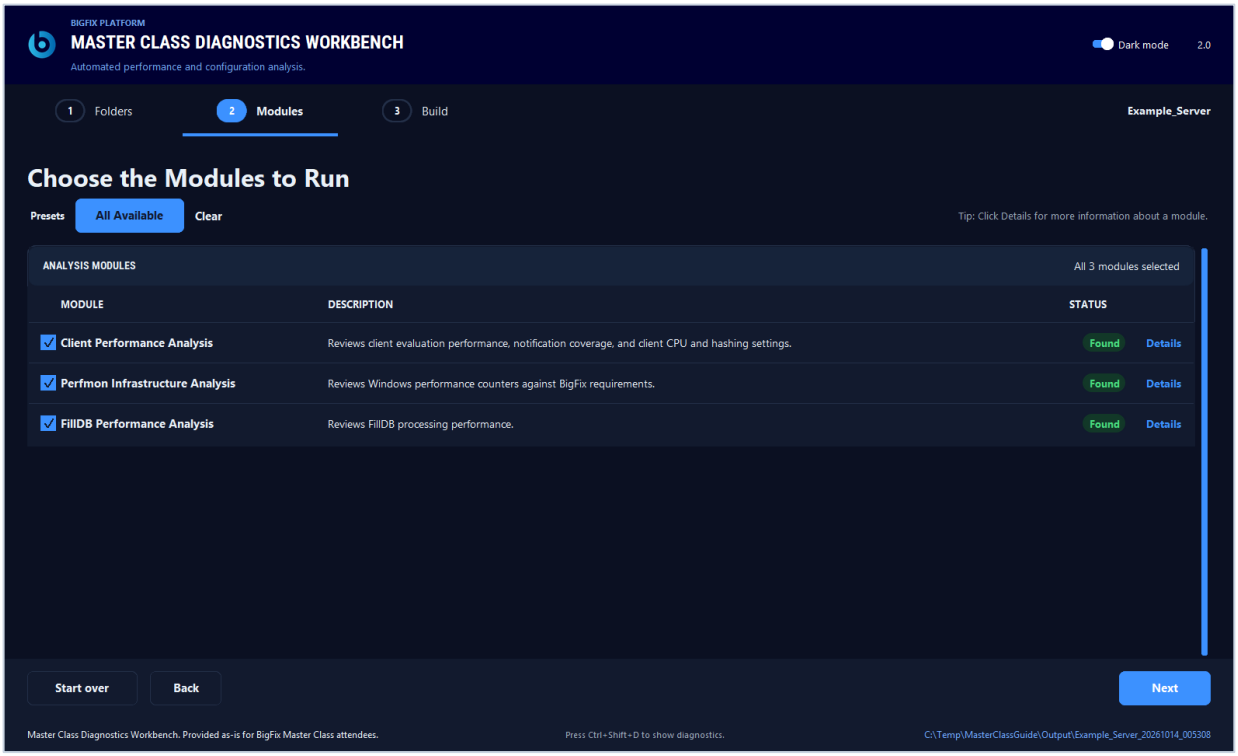


Figure 2. Step 2 with the three analysis modules in dark mode.

Control	Action
All Available	Selects the modules with available inputs.
Clear	Deselects all modules.
Choose or Change	In the Client Performance Analysis and FillDB Performance Analysis input tables, **Choose** appears when no valid input is selected. **Change** appears when an input is already found.
Choose folder or Change folder	Selects or replaces a folder input.
Browse	Selects a capture file in a Perfmon capture panel.

Select at least one module. For date fields, use the calendar control or enter **MM-DD-YYYY**. Where a time is needed, use **MM-DD-YYYY HH:MM**. **Next** and **Build** also validate these formats.

5.2 Statuses

Status	Meaning	Action
Found	The input was found. For a selected module, the required review values are filled in.	Continue.
Needs A File	A usable required input has not been selected.	Open **Details** and choose the input.
Needs Review	Required values are incomplete.	Open **Details** and fill them in. The status updates automatically.
Scanning	Input detection is in progress.	Wait for the scan to finish.
Not Scanned	Inputs have not yet been scanned.	Select a Data Folder on Step 1, or choose the input manually.

5.3 Client Performance Analysis

Item	Detail
Required input	A Client Loop XML or CSV capture, or a Client Performance Counters Basic export. Supported primary file extensions are **.xml**, **.csv**, **.xlsx**, **.xlsm**, and .xls. The Advanced export is not supported. The Deployment_Information folder from the Collect Deployment Information task is also required.
Required values	Report Start Date and Report End Date for the collection being analyzed.
Optional inputs	Top 10 slowest evaluations and UDP notifications.

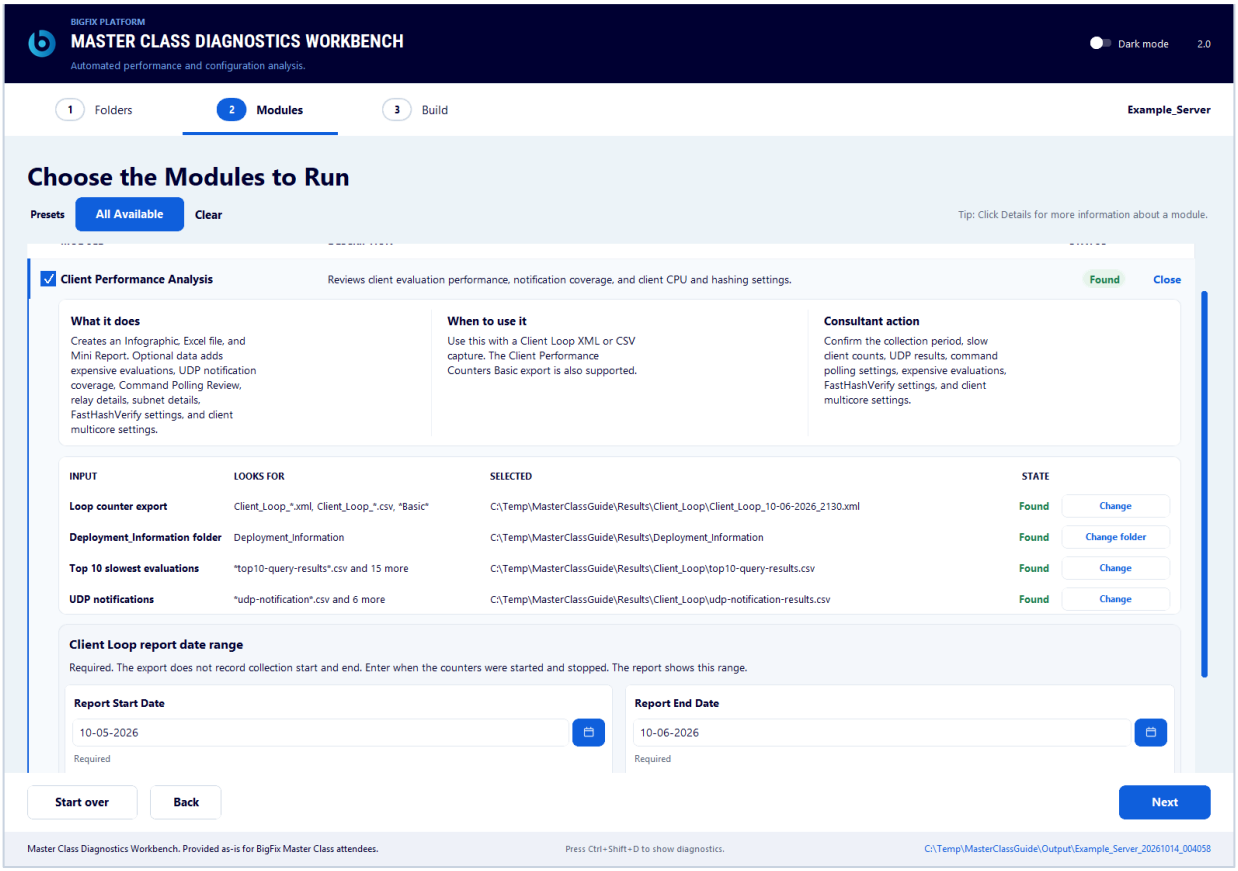


Figure 3. Client Performance Analysis Details with the input and report dates.

5.4 Perfmon Infrastructure Analysis

The Details view has two capture panels: **Capture 1 - BigFix Root Server** and **Capture 2 - SQL Server host**.

Item	Detail
Required input	Capture 1 **.blg** file.
Required values	**CPUs**, **Disks**, and **RAM** must contain positive whole numbers. CPUs and disks are detected from the capture. Enter RAM in GB manually. Set **Virtual machine** to match the server. An unchecked Virtual machine checkbox means the capture is treated as a physical server.
Optional inputs	Capture 2 **.blg** file, with its own positive CPU, disk, and RAM values. Capture 2 must be a different file from Capture 1. Collection start and end can be left blank.

Capture 2 control	Action
Re-detect	Detects the CPU and disk values from the capture again.
Browse	Selects the capture file.
Remove	Removes the second capture.

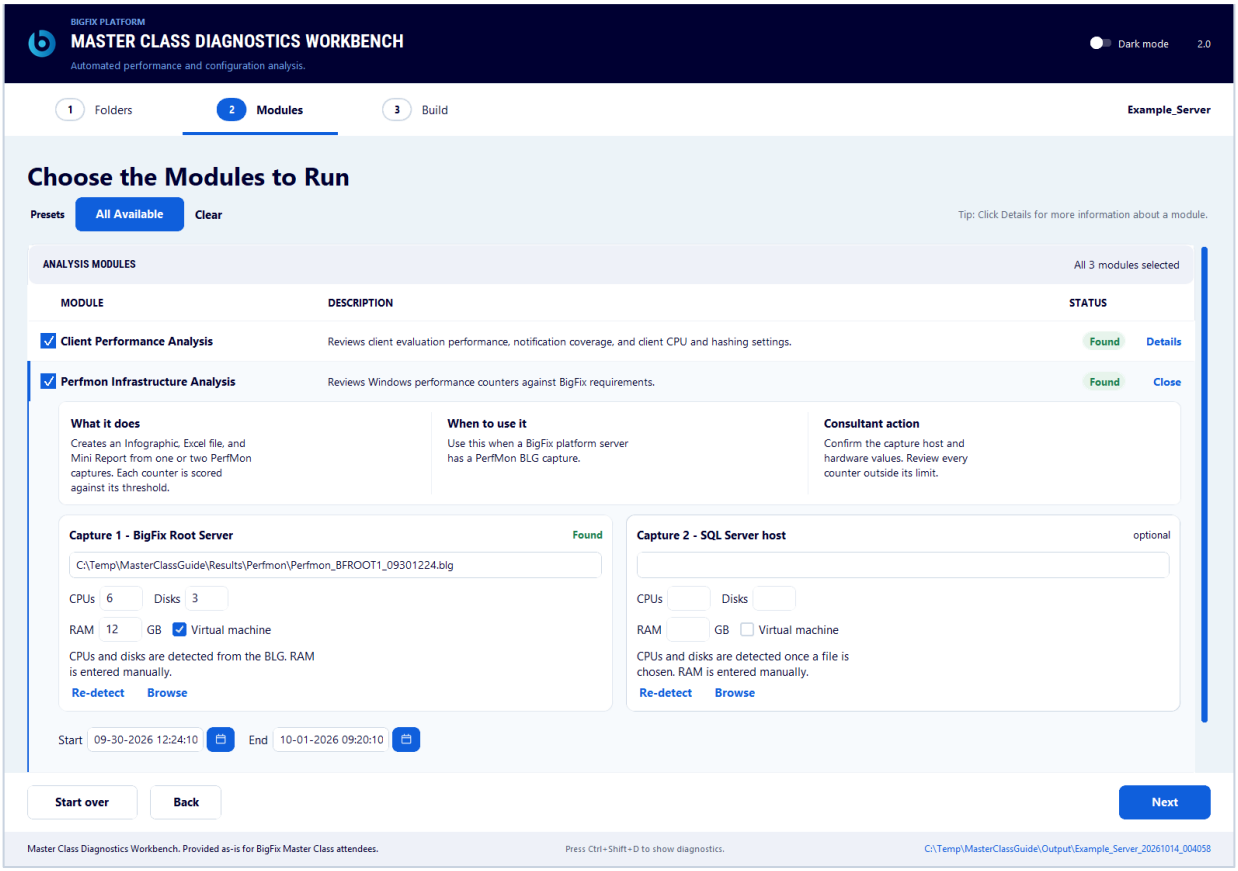


Figure 4. Perfmon Infrastructure Analysis Details with the capture and hardware controls.

NOTE Perfmon analysis requires the Windows **relog.exe** utility.

5.5 FillDB Performance Analysis

Item	Detail
Required input	Raw logs: Supply one log or the complete set, including rollover files. Keep the logs together in one folder and select any log in the set. The Workbench combines the related logs in chronological order and runs MXFillDBPerf for you. Analyzed CSV: Supply a .csv file generated by MXFillDBPerf from your logs. You must have run the utility with --healthcheck so the CSV includes the required health-check analysis.
Optional inputs	Collection start and end.

NOTE For raw logs, the Workbench uses the bundled Utilities\MXFillDBPerf.exe with --healthcheck automatically. An existing CSV is used as supplied; the Workbench does not rerun the utility for it.

SECTION 06

Step 3: Build

STEP 1 Folders	STEP 2 Modules	STEP 3 Build

Step 3 shows the run details, builds the selected modules, and opens the results.

1. Check the run name, selected modules, and output folder in **Run Details**.
2. Select **Build**.
3. Monitor the module statuses. Modules run sequentially. Large Perfmon captures can take several minutes.
4. Use **View Logs** to inspect progress or errors.
5. Select **Open Output** to open the run folder.

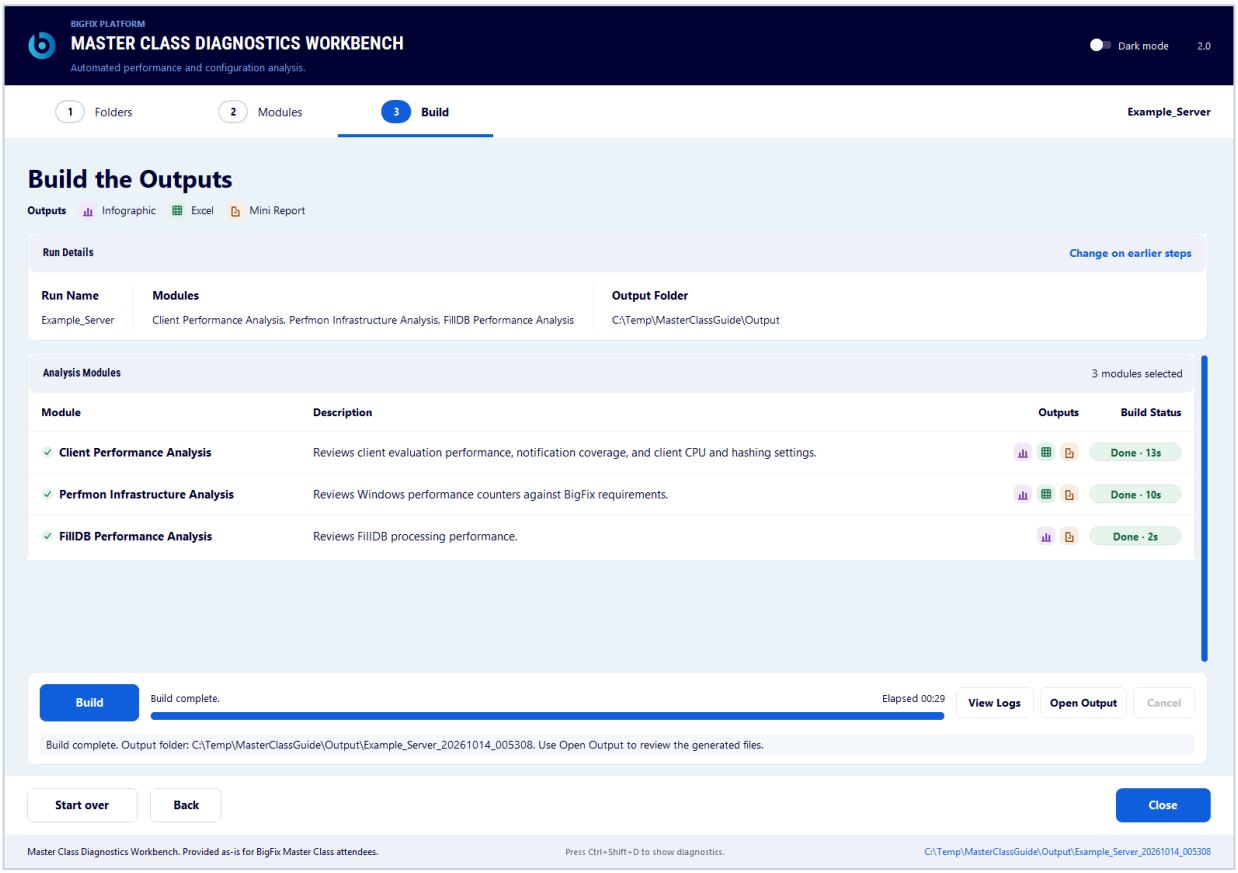


Figure 5. Step 3 after a successful build.

Select **Cancel** to request cancellation of a running build. A canceled build can leave incomplete outputs in the run folder.

6.1 Output Terms

Output	Meaning
Mini Report	The narrative analysis (**.docx**).
Infographic	The visual summary (**.png**).
Excel workbook	Supporting data and detail (**.xlsx**).

6.2 The Run Folder

Each build creates a folder in the Output Folder, named from the run name and a timestamp. The selected modules write into these subfolders.

Module	Subfolder	Main outputs
Client Performance Analysis	Client_Loop	Mini Report (**.docx**), Infographic (**.png**), and Excel workbook (**.xlsx**).
Perfmon Infrastructure Analysis	Perfmon	Mini Report (**.docx**), Excel workbook (**.xlsx**), and a **_Perfmon_Charts** folder with the scorecard and counter charts. A second capture adds its own workbook and chart folder.
FillDB Performance Analysis	FillDB_Performance	Mini Report (.docx) and Infographic (.png). The analyzed data is included in the Mini Report. No Excel workbook is needed because MXFillDBPerf does not produce a separate dataset for sorting or filtering.

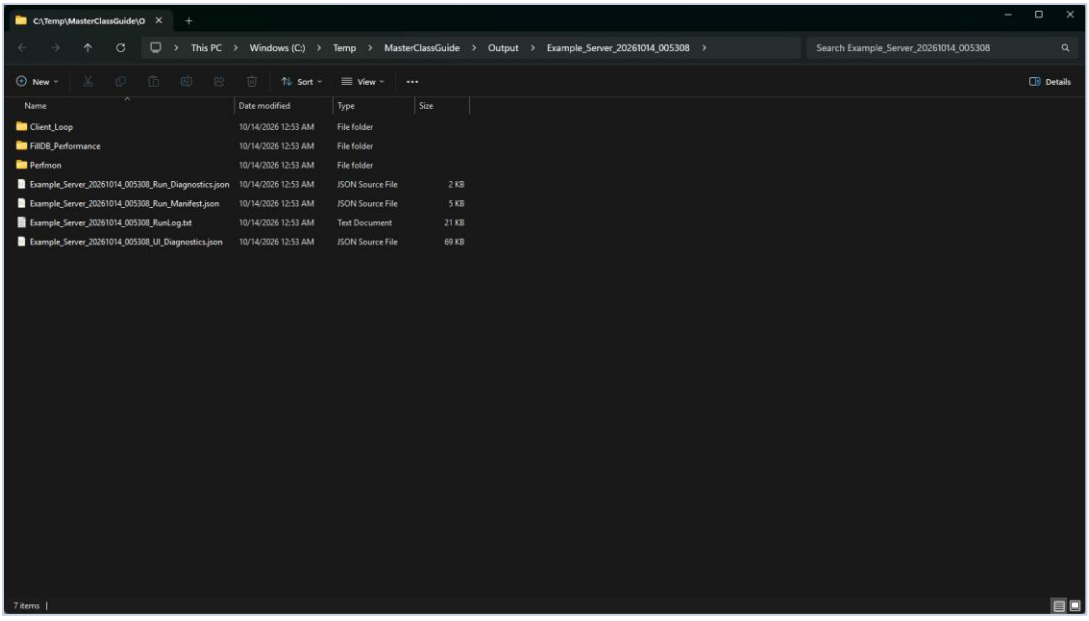


Figure 6. A generated run folder with the module subfolders.

A successful run also writes a run log and a manifest. A failed or canceled run might not contain every expected file.

6.3 Estimated Build Time And Memory

Use these example times to plan a build in Master Class Diagnostics Workbench 2.1 with Client Performance, PerfMon and FillDB selected. Reference workstation: Windows 11 laptop with an Intel Core i9-12900H, 64 GB RAM and a local NVMe SSD. Benchmark date: 20 October 2026.

Managed endpoints	Estimated build time	Peak report memory (GiB)
50,000	2 min 16 sec	1.7
100,000	4 min 19 sec	2.7
150,000	6 min 12 sec	3.7
200,000	8 min 15 sec	4.6
250,000	10 min 16 sec	5.5
300,000	12 min 07 sec	6.7

For a comparable 300,000-endpoint build, aim for about 10 GiB of available RAM after selecting the data folder and opening other required applications. This allows headroom above the measured peak; it is planning guidance, not a tested minimum computer specification.

Memory figures cover the report processes, not total computer use. Times exclude setup and user interaction. Tests used a six-hour PerfMon capture; longer captures and larger logs can take more time and memory.

SECTION 07

Troubleshooting

Problem	Action
The launcher cannot find a supported Python version.	Install Python 3.12 or newer and make it available through the Python launcher or PATH.
First setup fails.	Read the launcher error. Check write access and package access. For incomplete setup, close the Workbench, remove its **.venv** folder, and launch again.
A module shows Needs A File.	Select the module, open **Details**, and choose a supported input.
A module shows Needs Review.	Enter the Client Performance Analysis report dates, or complete the Perfmon CPU, disk, and RAM values.
The build reports an error.	Open **View Logs** and inspect the reported input or processing error.
An Infographic is missing.	Confirm that Microsoft Edge or Google Chrome is installed, and inspect **View Logs**.
A Perfmon input cannot be processed.	Confirm that the Workbench is running on Windows, the input is a valid **.blg** file, and the hardware values are complete.

NOTE For a slow or unresponsive run, keep the run log and any generated ***_Run_Diagnostics.json** and ***_UI_Diagnostics.json** files for troubleshooting.

SECTION 08

Security And Data Handling

8.1 Air-Gapped Analysis

After setup, the Workbench can run in an air-gapped environment. It reads collected files and writes reports locally, without connecting to live BigFix or SQL systems, uploading customer data, or sending telemetry. Collection is a separate activity that takes place before analysis.

8.2 The Private Python Environment

The launcher installs the runtime packages listed in requirements.txt into a private .venv folder. Your global Python installation and its packages are not changed. Complete package installation before disconnecting the workstation for air-gapped use.

To rebuild the environment, close the Workbench, remove its .venv folder, and launch again. Rebuilding requires access to the package source again.

8.3 Dependency Cooldowns

Runtime dependencies are pinned to specific versions. Dependency cooldowns hold back newly published package versions for at least seven days before routine updates use them. This gives time for problems with a new release to be identified before adoption.

8.4 Code Sweeps

The project uses automated code sweeps covering tests, code quality, static security analysis, known dependency vulnerabilities, and license inventory. The sweep workflow also supports secret scanning. These are maintainer checks, separate from running an analysis.

Release packages include a CycloneDX Software Bill of Materials (SBOM), third-party license notices, a manifest with SHA-256 checksums for the packaged files, and a checksum for the release download. The Python source is available under the MIT license for inspection.

8.5 Data Handling Rules

- Treat captures and generated files as confidential.
- Use sanitized examples in screenshots.
- Review collected files and generated outputs before you share them.
- Five tasks change BigFix client settings on the computers they target: Enable Client CPU Multicore Scaling, Enable Client FastHashVerify Setting, Set CPU Utilization to 2% on Windows Multi-Core Systems, Set CPU Utilization to 20%, and Set Command Polling to 10 Minutes over Average Evaluation Time. A setting stays in place until it is changed again. No task that resets these settings is supplied.
- The Excel workbooks list individual computers and network details such as relay names and subnets. Share them only with the customer whose deployment they describe.
- The Collect tasks take the console user name and password as BigFix secure parameters. Use a dedicated Master Operator account for the collection and change its password afterward.